

MMWAVE DFP User Guide



Product Release 2.4.18.1

Contents

Abbreviations	3
1. System Overview	4
1.1 mmWave Firmware.....	4
1.2 mmWaveLink	4
2. xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC - System Deployment.....	7
2.1 Mailbox Message Scheme.....	7
3. Getting started	9
3.1 Requirements and Build Settings.....	10
3.2 Advance Frame and Advance Chirp Configuration	10
3.3 Chirp Parameter and Chirp Quality Data Extraction and Verification	10
3.4 Dynamic Chirp Test	11
3.5 Monitoring Test.....	11
3.6 RF Program Filter Test.....	11
4. PC Connection.....	11
4.1 Connection with xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO//xWR2x4xLC EVM	11
5. API Sequence	12
6. mmWave Sensor Advanced Features	14
6.1 Advanced frame configuration	14
6.2 Advanced chirp configuration.....	17
6.3 Dynamic chirp configuration.....	26
6.4 Dynamic profile configuration	28
6.5 Dynamic per chirp phase shifter configuration	29
6.6 Factory Calibration.....	29
6.7 CP/CQ Data Extraction and Verification for xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC devices.....	31
7. Platform and Device Support.....	37

Abbreviations

Abbreviation	Description
AE	Asynchronous Event
DFP	Device Firmware Package
RadarSS	Radar Sub system
MSS	Main Sub system
DSPSS	Digital Signal Processing Sub System
SPI	Serial Peripheral Interface

1. System Overview

The mmWave device firmware package (DFP) is split in three broad components: mmWave Firmware, mmWaveLink and mmWave RF evaluation.

1.1 mmWave Firmware

mmWave Firmware is responsible for configuring RF/analog, digital front end in TI mmWave radar devices and consists of below component:

- Radar SS firmware

All the services of Radar SS firmware are available to Main SS/DSP SS using the APIs in mmWaveLink framework.

mmWave Firmware binaries are available at '**`mmwave_dfp_<ver>\firmware\`**'

Note: DSP core is not available on xWR254x, xWR294xLC

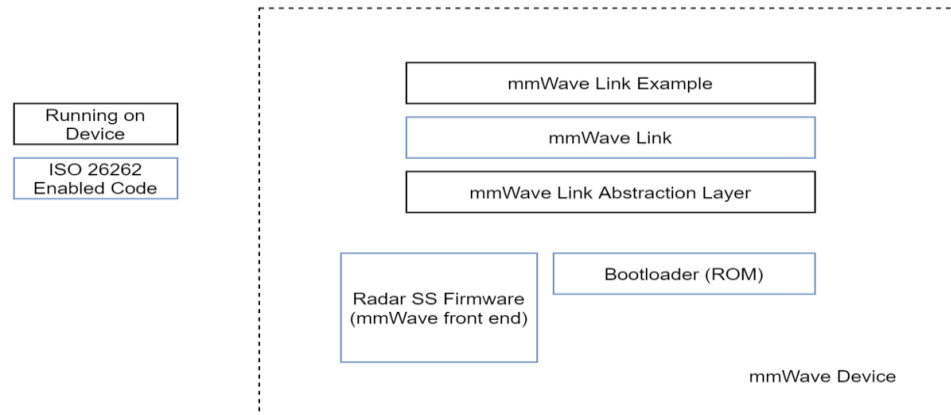
1.2 mmWaveLink

1.2.1 mmWaveLink framework

mmWaveLink framework acts as driver for Radar sub-system. It exposes a suite of low-level APIs which allow application to enable, configure and control the Radar SS. It provides a well-defined platform and OS abstraction for the application to plug in the communication driver and OS routine callbacks to communicate with the TI mmWave devices. Main SS/ DSP SS can use this framework to connect and configure xWR294x/xWR245x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC device.

mmWaveLink source and library are available at '**`mmwave_dfp_<ver>\ti\control\mmwavelink`**'

Figure-1: Depicts high level modules of

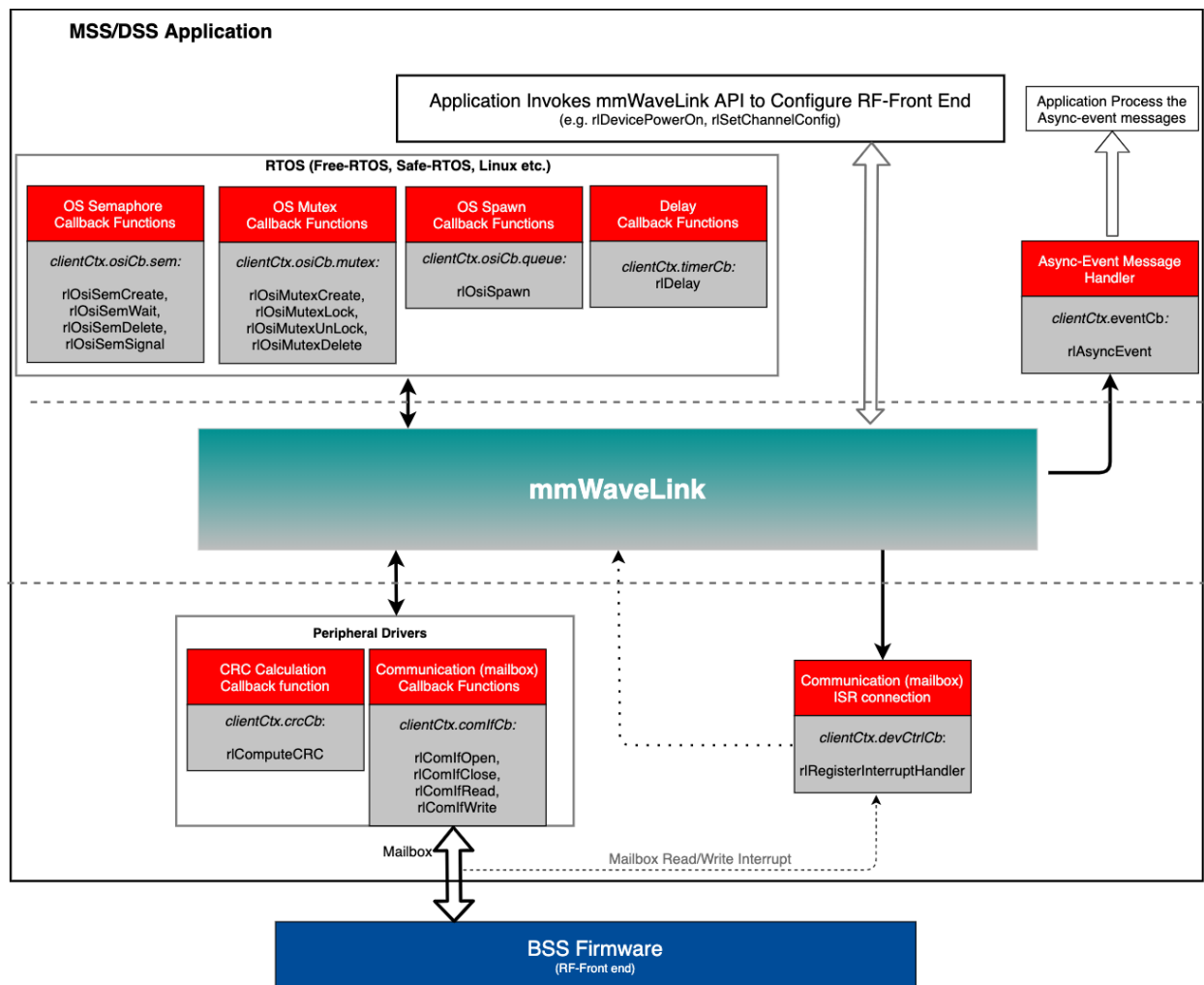


xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC device.

mmWaveLink features:

- Portable framework which abstracts the services provided by the mmWave device.
- Provides low level APIs to control the mmWave device and RF modules
- Abstracts communication protocol with the mmWave device
- Handles communication errors, Notifies exceptions
- Is platform independent
- Is OS agnostic, can work in single threaded (non OS) environment as well
- On AWR2243, it resides in External MCU and communicates over SPI
- For AWR294x/AWR1843 will reside in the internal Cortex R5F/R4F or DSP and communicates over Mailbox

Application needs to provide access of OS features, peripheral and interrupt to mmWaveLink via different set of callbacks as shown in below picture.



Note: xWR294x/xWR245x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC mmWaveLink library and source files are part of mmWave DFP v2.4.x (under mmWave mcuplus SDK installation)

1.3 mmWave RF Evaluation

For mmWave RF/System evaluation purpose, mmWave Studio/Radar studio Tool which is designed to communicate with all variants of TI mmWave devices for RF and system performance evaluation can be used; this tool is available at <http://www.ti.com/tool/MMWAVE-STUDIO>.

The RF evaluation firmware is available at '**mmwave_dfp<ver>lr_eval\rf_eval_firmware**' directory which should be used in SOP-2 (development) mode only. Refer to mmWave Studio user guide for more details.

2. xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC System Deployment

The device provides a mailbox mechanism to asynchronously exchange the messages between any two processors. Each processor has a mailbox memory space, and registers designated to be used by other processor that wishes to communicate with it. Each processor has two sets of mailbox memory space and registers, and each set is designated per other processor to communicate with it.

2.1 Mailbox Message Scheme

The processor which wishes to send a message to another processor writes the message to the mailbox memory space, then interrupts the receiver processor. The receiver processor acknowledges the interrupt, then reads the message from the mailbox memory space. The receiver informs the sender that the message is read by an interrupt, which is acknowledged back by the sender. The sender must not initiate another message to the same receiver until the previously initiated mailbox interaction with the same receiver is complete.

- SENDER writes the message in the RECEIVER mailbox.
- SENDER triggers an interrupt to RECEIVER by writing 1 to `<SENDER_SS>_CTRL:<SENDER>_MBOX_WRITE_DONE [RECEIVER]`. Note: It is writing to its own CTRL space.
- RECEIVER gets a single interrupt for all interprocessor communication, which is an aggregated interrupt.
- RECEIVER reads the register `<RECEIVER_SS>_CTRL::<RECEIVER>_MBOX_READ_REQ` and sees bit [SENDER] is 0x1.
- RECEIVER waits for some 60-70 clock cycles before going to next step.
- RECEIVER writes to 0x1 to `<RECEIVER_SS>_CTRL::<RECEIVER>_MBOX_READ_REQ [SENDER]` to clear the interrupt.
- RECEIVER reads the message.
- RECEIVER writes to 0x1 to `<RECEIVER_SS>_CTRL::<RECEIVER>_MBOX_READ_DONE_ACK[SENDER]` to generate an acknowledgment interrupt to SENDER.
- SENDER gets a single interrupt for all interprocessor communication, which is an aggregated ACK interrupt.
- SENDER reads the register `<SENDER_SS>_CTRL:<SENDER>_MBOX_READ_DONE` and sees bit [RECEIVER] is 0x1.
- SENDER waits for some 60-70 clock cycles before going to next step.

SENDER writes 0x1 to `<SENDER_SS>_CTRL:<SENDER>_MBOX_READ_DONE [RECEIVER]` to clear the interrupt.

Refer to the Mailbox driver + mmwavelink communication implementation in the mmWave MCU Plus SDK for more information.

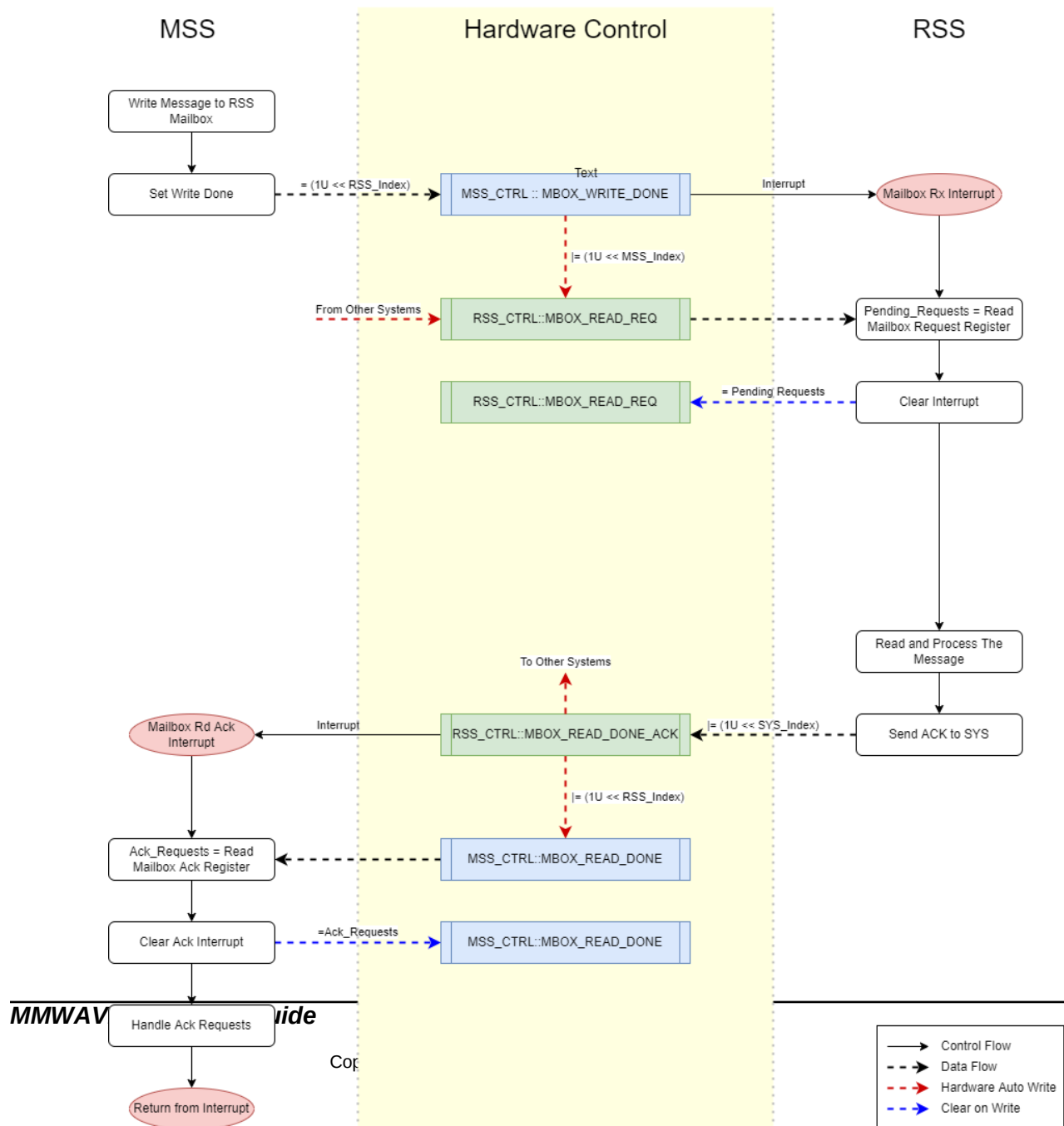


Figure 2

3. Getting started

This is test application where it uses direct mmwavelink APIs to configure BSS of xWR294x/xWR254x/ xWR2x4xP/xWR2x4xECO/xWR2x4xLC over mailbox. The best way to get started with the mmWave DFP is to start running mmwaveLink example project that is provided as part of the package. The example provides the opportunity to get accustomed to the mmwavelink APIs and to explore various features of the device. You can run the example on either the MSS or DSS core. The main function has an amalgamation of various test cases which are set to false by default. The user can set various test cases as true to test the different features in the example. Also, most of the APIs usage of mmwavelink framework is covered in the example project and the documentation for all the APIs is available in the Interface Control Document (ICD) present in the DFP package itself under the “docs” folder. Some features of the

device, currently covered in the examples are described as below. It is recommended to run only one feature at a time in this example project.

NOTE: mmwaveLink example is located at
mmwave_mcuplus_sdk_<ver>\mmwave_mcuplus_sdk_<ver>\ti\control\mmwave\link_test

3.1 Requirements and Build Settings

Please install the latest MCU plus SDK package in your system. Before running this test example, it is required to set the environment variables (by running the setenv.bat file from the SDK package in a command prompt) and build the example. Please ensure that all the variables in the setenv.bat file are mapped correctly. For example the “MMWAVE_SDK_DEVICE” variable should be set to “awr2944” for xWR294x device, “awr2544” for xWR254x device and “awr2x44P” for xWR2x4xP, xWR2x4xECO, xWR2x4xLC device. Once you set the environment variables you can build the example. You will need to change path in your command prompt to the folder which contains the test example and give the build command (gmake all).

3.2 Advance Frame and Advance Chirp Configuration

The example provides two Boolean variables in the main_mss.c or main_dss.c to enable/disable this feature. By default, it is disabled and the user will need to enable them in the source file to see this functionality. The two variables are gLinkAdvanceFrameTest and gAdvChirpTest. Please refer to [section 6](#) for more details on advance frame and advance chirp configuration and the code flow. The idle time is varied by adding a dither value and example verifies that whether the idle time increased or not for the chirps and prints out the result.

3.3 Chirp Parameter and Chirp Quality Data Extraction and Verification

The example provides a Boolean variable in the main_mss.c or main_dss.c to enable/disable this feature. By default, it is disabled and the user will need to enable the variable to see the functionality. The variable is cqDataVerifyTest. Currently the xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC device only supports CQ2 data and the CP information. The example enables the RX Saturation monitor to enable the data collection for CQ2 and then transfers this data to L3 memory for further processing via an EDMA driver. The data can then be verified. Currently we do not use any other device as an interference source and hence the RX saturation monitor will show all data as zeroes as there are no saturation events. If you add an interference source you will non-zero values. Please refer to the ICD (Interface Control Document) for more information on CP and CQ data. Please refer to [section 6.7](#) to get more information on how to enable this monitoring and how to collect and verify this data.

3.4 Dynamic Chirp Test

The example provides a Boolean variable in the main_mss.c or main_dss.c to enable/disable this feature. By default, it is disabled and the user will need to enable the variable to see the functionality. The variable is gDynamicChirpTest. This API can be used to dynamically change the chirp configuration while frames are on-going. The configuration will be stored in software and the new configuration will be applied after receiving the AWR_DYN_CHIRP_ENABLE_SB API. To verify that new Chirp configurations have been taken effect on RF-front end, we need to take time comparison of chirp interrupt arrival before and after dynamic chirp config. As in DynChirp Config Chirp-Idle Time has increased so after this come in effect, every chirp interrupt arrival will have extra time delay. Please refer to [section 6.3](#) for more details on this.

3.5 Monitoring Test

The example provides a Boolean variable in the main_mss.c or main_dss.c to enable/disable this feature. By default, it is disabled and the user will need to enable the variable to see the functionality. The variable is gLinkMonitoringTest. AWR monitoring can be configured through a set of API sub blocks defined in this section. Note that these APIs cover the RF/Analog related monitoring mechanisms. There are common configuration APIs that control the overall periodicity of monitoring, as well as, enable/disable control for each monitoring mechanism. Then, for each monitoring mechanism there is an individual API to allow the customer to set an appropriate threshold for declaring failure from that monitoring. Also, for each monitoring mechanism, there is an individual API to report soft (raw) values from that monitoring. Refer Monitoring application note and the ICD for more info on exact use case and threshold configuration recommendations.

3.6 RF Program Filter Test

The example provides a Boolean variable in the main_mss.c or main_dss.c to enable/disable this feature. By default, it is disabled and the user will need to enable the variable to see the functionality. The variable is gLinkProgFiltTest. This sub block can be used to program the coefficients for the external programmable filter. The programmable filter allows for a trade-off between digital filter chain settling time and close-in anti-alias attenuation. Please refer to the ICD for more details.

4. PC Connection

4.1 Connection

with

xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO//xWR2x4xLC EVM

When the EVM is powered on and connected to Windows PC via the supplied USB cable, there should be two additional COM Ports in Device Manager as shown in Figure-5. See EVM User Guide for details on the COM port.

If below COM ports doesn't show up in the Device Manager install the emupack available [here](#).

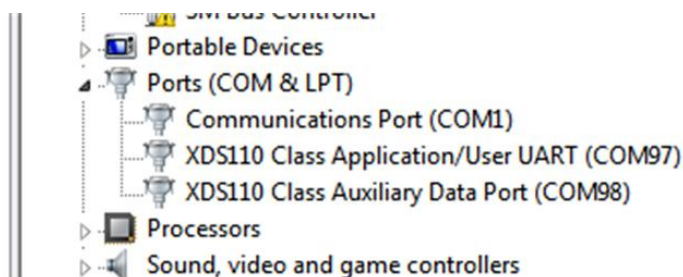


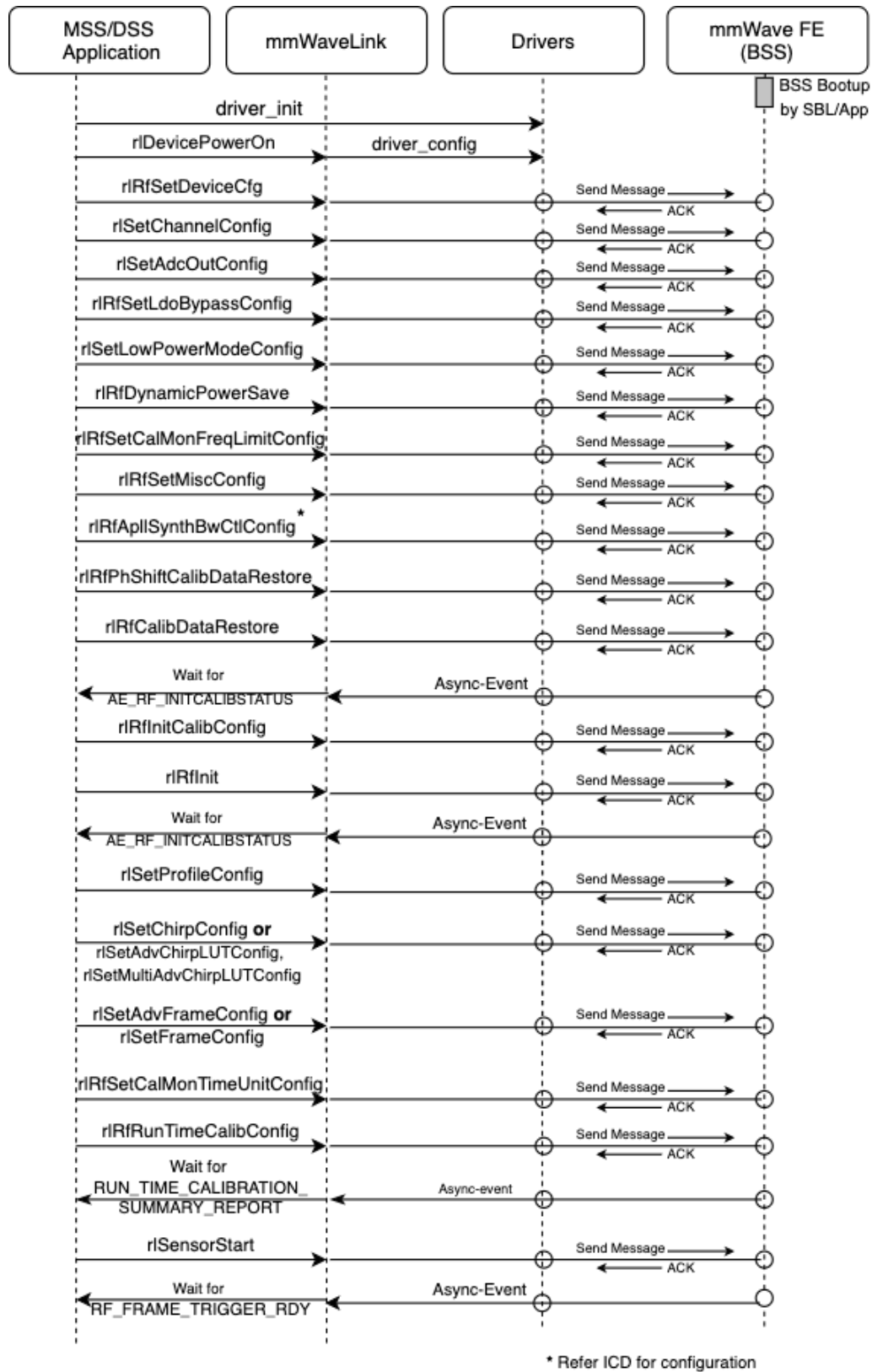
Figure 3. xWR294x /xWR254x /xWR2x4xP /xWR2x4xECO /xWR2x4xLC EVM COM Ports

The USB connection provides below functionalities.

- JTAG for CCS connectivity – Only required for debug purpose. More detail on this is not in the scope of this document.
- UART for downloading firmware in serial data flash using UniFlash. Refer to respective user guides for more details.

5. API Sequence

Below figure shows the API sequence in mmwavelink_example (default setting). For more details on these APIs, refer to mmWaveLink framework doxygen HTML files in '**mmwave_dfp_<ver>\ti\mmwavelink\docs**' directory. Please note that the RF bootup should be done by the SBL and the example assumes that the bootup is complete.


Figure 4

6. mmWave Sensor Advanced Features

TI's mmwave sensor devices provide large flexibility in configuring chirp parameters. Many complex use-cases require dynamic reconfiguration of device within a frame or while device is framing. Below mentioned APIs can be used to cater those use-cases. The usage of advanced APIs is explained based on multiple possible use-cases we have come across but there are still other possibilities which may not covered here. These APIs are explained with the all required timing and existing limitation w.r.t. different device variants.

This can referred on top of another app-note (<http://www.ti.com/lit/an/swra553/swra553.pdf>) which explains the fundamental programming chirp parameters in TI mmwave sensor device.

6.1 Advanced frame configuration

mmWave sensor provides an advanced option for frame configuration. In this configuration, user can configure multiple type of frames compare to legacy frame configuration where only one type of frame is permitted. Advanced frame configuration allows up to four sub-frames, with multiple bursts in each sub-frame.

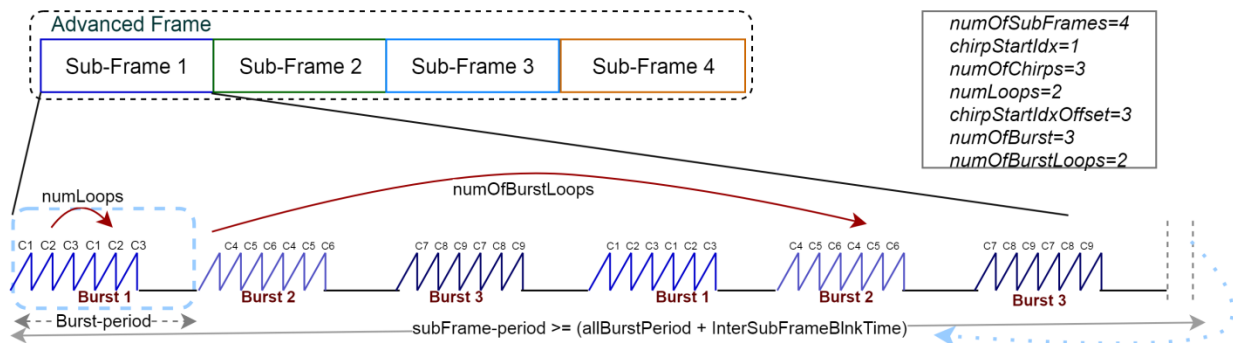


Figure 5. Example of Advanced Frame

As you have seen that based on the desired application the chirp configurations need to be set differently. But what if we need to support multiple modes, for example ultra-short range, short range and/or mid-range, simultaneously using a single radar device? The advanced frame configuration available in TI's mmwave sensor allows for large flexibility to have multiple chirp configurations in a single frame. The frame can be constituted using a sequence of "sub-frames" with each of these sub-frames representing a different radar mode. The *rlSetAdvFrameConfig* API helps enable this kind of configuration.

To provide maximum flexibility of the chirps within a frame, the advanced frame configuration API provides the ability to break a frame into different sub frames (up to 4). Each of the sub frames consists of multiple bursts of chirps (up to 512 bursts). Each burst can consist of 512 unique chirps (but can be looped over these chirps to generate a larger sequence) which are associated to one of the 4 profiles, and the start chirp index can be programmed to have a fixed

offset from the previous burst. A set of bursts in a sub-frame can be further looped in software up to 64 times. The below figure is an example how a sub frame is formed.

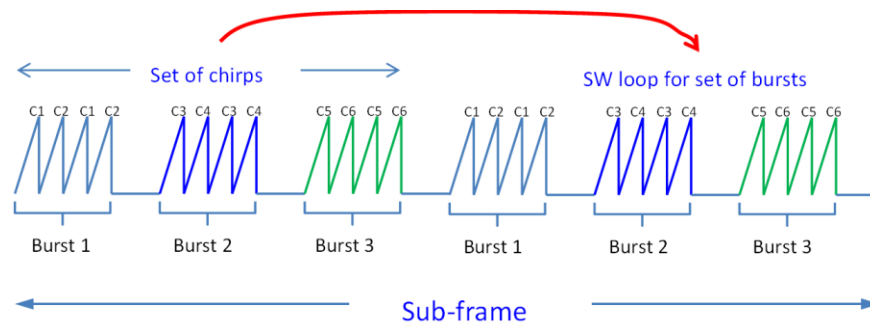


Figure 6. Sub Frame Structure Showing Three Bursts Looped Twice (start index having an offset of 2)

The frame is formed by a maximum of four such sub frames, and each of the sub frames can have a different set of chirps. The different chirps can also use different transmitters (which could possibly have different antenna configurations).

The timing requirements for the advanced frame are as follows: Inter-burst time should be $\geq 50 \mu\text{sec}$, inter sub-frame time should be $\geq 100 \mu\text{sec}$ and inter frame time should be $\geq 200 \mu\text{sec}$.

Note- all the variable/parameter used here to explain the concepts are based on mmWaveLink.

Creating a Burst

A burst can have max of 512 unique chirps in it but can be looped over these chirps to generate a larger sequence. In a burst, index of chirp starts with `chirpStartIdx` and index increments till `numOfChirps`, this chirp sequence can be repeat by `numLoops` in a single burst. In the next burst, variation in chirp index can be done by setting `chirpStartIdxOffset`. As depicted in last figure, chirp index for the 2nd burst will be 4 when '`chirpStartIdxOffset=3`'.

[next_burst_chirp_start_idx = last_burst_chirpStartIdx + chirpStartIdxOffset]

Burst Periodicity

Burst periodicity should be accounted with extra blank time along with total chirp time. User needs to set the burst periodicity based on calculation below to avoid any error where *InterBurstBlankTime* is required for sensor calibration/monitoring.

*[BurstPeriodicity $\geq$ (numOfChirp * numLoops) + InterBurstBlankTime]*

Sub-Frame Periodicity

While setting the sub-frame periodicity in advanced frame user needs to provide extra time [*InterSubFrameBlankTime*] which is required for sensor calibration/monitoring, transferring out

any safety data, hardware reconfiguration and trigger the next sub frame. Sensor may throw an error while triggering the advanced frame, if sufficient blank time is not provided.

$$[BurstPeriodicity \geq (numOfChirp * numLoops) + InterBurstBlankTime]$$

Scenario	InterSubFrameBlankTime (usec)
Default	≥ 100
Loopback configuration enable	≥ 350
Test source configuration enable	+150

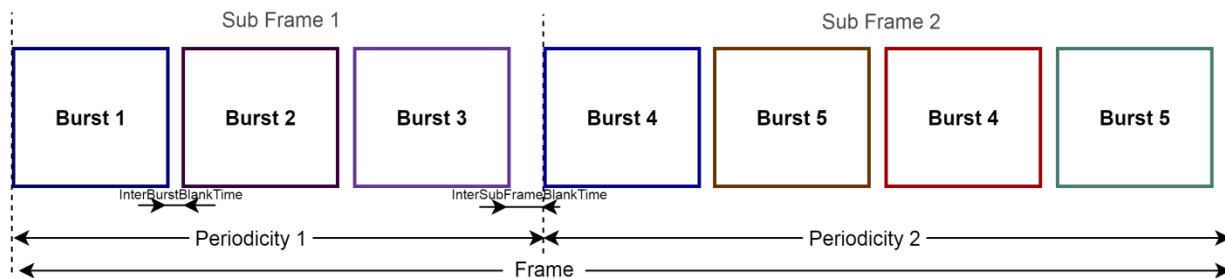


Figure 7. Example of Advanced Frame Configuration of Two Sub Frames

Override the existing Chirp Profile Index

It may possible that each chirp is connected to different profile index as set by `rlSetChirpConfig` API. So with this condition, chirps comprised in a sub-frame may be connected to different profile index. But advanced frame API provides an option to unify profile index in a sub-frame, i.e. all the chirps in a sub-frame can be connected to single profile index.

Set `rlAdvFrameSeqCfg_t->forceProfile` to '1' which enables the option to select specific profile index for the burst and set `rlFrameCfg_t->forceProfileIdx` to required profile index for that sub-frame. It will override the profile-ID selection for all the chirps in sub-frame/burst.

NOTE:

- This setting won't overwrite the profile-index in chirp RAM, it will be effective only during this sub-frame.

Loopback configuration

Refer [section](#) for Loopback APIs.

Sub-Frame Trigger

By default, all the configured sub-frame will be triggered internally by the device. But if some use-case requires triggering the sub-frame manually, then RadarSS provides software and

hardware triggered mode options in advanced frame to trigger each sub-frame. Host needs to invoke `rlSensorStart` API if SW triggered mode is set, else provide `SYNC_IN` pulse in HW triggered mode to set off each sub-frame. Host must need to provide these trigger events beyond last sub-frame boundary.

Following are the set of configurations for trigger option

Configuration	Action
<code>triggerSelect=1, subFrameTrigger=0</code>	<code>rlSensorStart</code> <u>only</u> at first frame.
<code>triggerSelect=1, subFrameTrigger=1</code>	<code>rlSetSubFrameStart</code> at every sub-frame boundary.
<code>triggerSelect=2, subFrameTrigger=0</code>	<code>SYNC_IN</code> pulse at every burst boundary.
<code>triggerSelect=2, subFrameTrigger=1</code>	<code>SYNC_IN</code> pulse at every sub-frame boundary.

6.2 Advanced chirp configuration

It provides the ability to program fixed delta increment for certain chirp parameters (chirp start frequency, idle time, phase shifter, etc.), on top of unique dithers selected from configurable look-up-table (LUT). The configurable look-up-table is an array of values loaded into a pre-configured "Generic SW Chirp Parameter LUT". The size of the generic LUT is 12KB and user has the flexibility to program any number of unique dithers for each chirp parameters. Thus the user can achieve fixed increment, or LUT based dither, or a combination of both.

Four types of control can be achieved on each parameters of a chirp

- Fixed value for all the chirps: To generate sequence of chirps which never changes, then only one value can be programmed in LUT ($P = 1$ and $K = 0$).
- Unique chirps: Index every K chirps in LUT to generate unique sequence of chirps.
- Delta increment every N chirps: On top of sequence of unique chirps from LUT, the fixed delta increment can be done every N chirps.
- The set of chirp parameters across bursts and sub-frames can be different by using offset to LUT.

There are total 10 chirp parameters that the advance chirp API can configure.

Chirp Parameter Index	Description
0	CHIRP PROFILE SELECT
1	CHIRP FREQ START VAR
2	CHIRP FREQ SLOPE VAR
3	CHIRP IDLE TIME VAR
4	CHIRP ADC START TIME VAR
5	CHIRP TX EN
6	CHIRP BPM VAL
7	TX0 PHASE SHIFTER

8	TX1 PHASE SHIFTER
9	TX2 PHASE SHIFTER
10	TX3 PHASE SHIFTER

It is highly recommend referring ICD document first to get the basic knowledge of this API. Here we exemplify this API using two different use-cases which explain many of complex concepts of this API.

Advanced chirp feature cannot be used with legacy chirp API as this API includes legacy API configurations along with additional complex features. Host needs to enable this feature first using `rIRfSetMiscConfig` API (`AWR_RF_MISC_CTL_SB`).

Example Illustration – 1

Delta Dither for CHIRP_FREQ_START_VAR:

DELTA_RESET_PERIOD = 0 (Same as RESET_MODE)
DELTA_PARAM_UPDATE_PERIOD = 1
SFO_CHIRP_PARAM_DELTA = 372830 (372830 * 3.6 GHz) / 2^26 = 20MHz

Total Chirp Freq = Legacy Start Freq + LUT Dither + Delta Dither

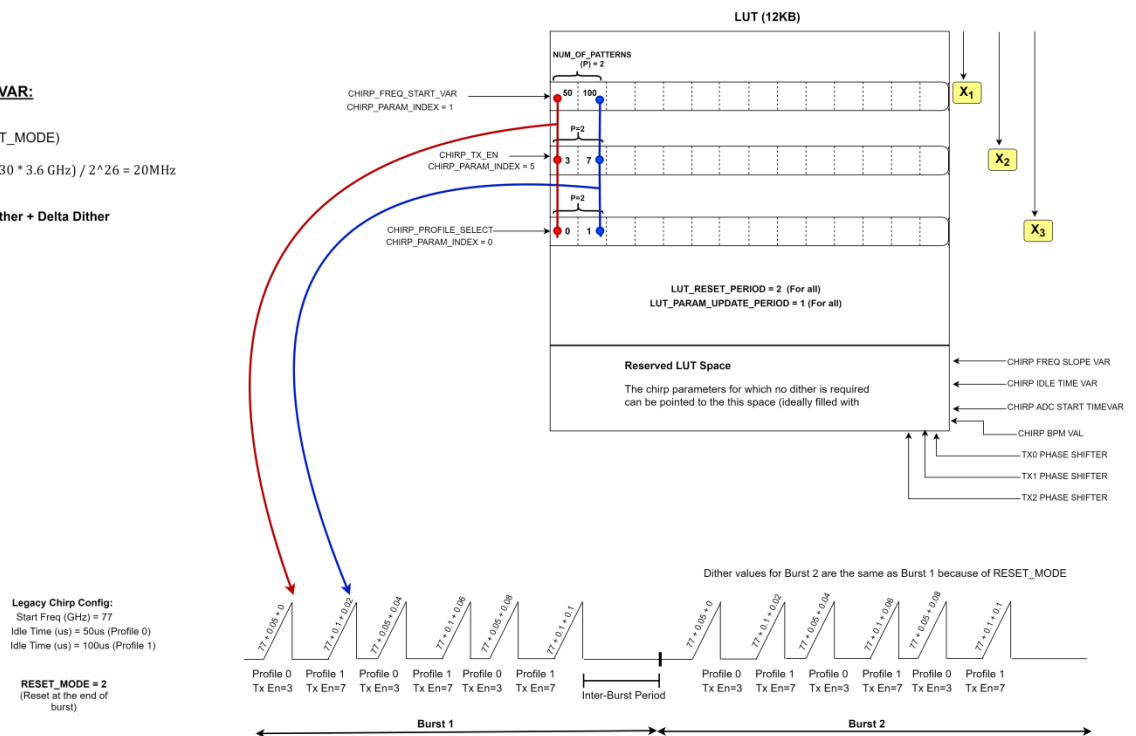


Figure 8. Advanced chirp – Example Illustration 1

In this figure, dither for 3 chirp parameters is considered:

- `CHIRP_TX_EN`
- `CHIRP_FREQ_START_VAR`
- `CHIRP_PROFILE_SELECT` (`CHIRP_PARAM_INDEX = 0`) : Start address offset = X_3

Rest of the chirp parameters for which no dither is required can point to the “Reserved LUT space” as shown in the figure. This space is ideally filled with zeros.

In the figure, *RESET_MODE* = 2 means Reset at the end of every burst. This is the reason why burst 1 and burst 2 are identical in the figure.

Now let's understand each of the three dithers one by one:

1. CHIRP_TX_EN (CHIRP_PARAM_INDEX = 5)

This Chirp Parameter dither is possible only through the LUT (Refer to the ICD).

- a. Start address offset of this chirp parameter in the LUT or the $LUT_PATTERN_ADDRESS_OFFSET = X_2$
- b. As it can be seen from the LUT, only two values are programmed for this chirp parameter, i.e. “3” and “7”, $NUM_OF_PATTERNS (P) = 2$. The values in the LUT denote 2 TX and 3 TX enabled simultaneously.
- c. Chirp 1 of Burst 1 takes the value from the 0th index of the LUT (Red marked arrow), whereas Chirp 2 of burst 1 takes the values from the 1st index (Blue marked arrow). Hence $LUT_PARAM_UPDATE_PERIOD (K) = 1$. This field updates the chirp parameter with the new value from LUT after every K chirps.
- d. It can be seen from the figure, although the default *RESET_MODE* = 2 (Reset at the end of every burst), reset of the LUT values is happening after every 2 chirps. The 3rd chirp shows the same value of *CHIRP_TX_EN* as the 1st chirp. This means, in addition to the default reset period, reset is happening after every 2 chirps from the LUT. Hence, $LUT_RESET_PERIOD (J) = 2$.
- e. *LUT_CHIRP_PARAM_SCALE* and *LUT_CHIRP_PARAM_SIZE* are not applicable for this Chirp Parameter (Refer to ICD).

2. CHIRP_PROFILE_SELECT (CHIRP_PARAM_INDEX = 0)

Again, this Chirp Parameter dither is possible only through the LUT (Refer to the ICD).

- a. $LUT_PATTERN_ADDRESS_OFFSET = X_3$
- b. Again, $NUM_OF_PATTERNS (P) = 2$ as only two values are programmed for this chirp parameter, i.e. “0” and “1”.

Definition of Profile 0:

Start Frequency (GHz) = 77

Idle Time (us) = 50

Definition of Profile 1:

Start Frequency (GHz) = 77

Idle Time (us) = 100

- c. Chirp 1 of Burst 1 takes the value from the 0th index of the LUT (Red marked arrow) and shows an idle time as 50us, whereas Chirp 2 of burst 1 takes the values from the 1st index (Blue marked arrow) and shows a higher idle time of 100us. Hence $LUT_PARAM_UPDATE_PERIOD (K) = 1$.
 - d. It can be seen from the figure, in addition to the default *RESET_MODE*, reset is also happening after every 2 chirps from the LUT. Hence, $LUT_RESET_PERIOD (J) = 2$.
 - e. *LUT_CHIRP_PARAM_SCALE* and *LUT_CHIRP_PARAM_SIZE* are not applicable for this Chirp Parameter also (Refer to ICD).
3. CHIRP_FREQ_START_VAR (CHIRP_PARAM_INDEX = 1)
This chirp parameter has both the types of dither, i.e. LUT and Delta.

LUT dither:

- a. $LUT_PATTERN_ADDRESS_OFFSET = X_1$
- b. $NUM_OF_PATTERNS (P) = 2$ as only two values are programmed for this chirp parameter, corresponding to “50MHz” and “100MHz”.
- c. The chirp parameter is updated with the new value from LUT after every 1 chirp, hence $LUT_PARAM_UPDATE_PERIOD (K) = 1$.
- d. In addition to the default *RESET_MODE*, reset is also happening after every 2 chirps from the LUT. Hence, $LUT_RESET_PERIOD (J) = 2$.
- e. $LUT_CHIRP_PARAM_SIZE = 0$, which corresponds to a size of 4 bytes in the LUT. Based on the frequency start value, this parameter needs to be set accordingly to accommodate within size (1/2/4 Bytes).
- f. *LUT_CHIRP_PARAM_SCALE* is not used in this case; hence its value is 0.

Delta Dither:

- a. $DELTA_RESET_PERIOD = 0$, i.e. same as *RESET_MODE* value (Reset at the end of every burst)
- b. Let's accumulate fixed delta dither every single chirp, hence $DELTA_PARAM_UPDATE_PERIOD = 1$
- c. In the figure, 20MHz of start frequency delta dither is accumulated every chirp, hence $SFO_CHIRP_PARAM_DELTA = 20MHz (372830 * 3.6 GHz) / 2^{26}$

To summarize,

Total Chirp Start Frequency = Legacy start frequency + LUT Dither + Delta Dither

	Chirp	Legacy Start Freq (GHz)	LUT dither (GHz)	Fixed delta dither (GHz)
Burst 1	0	77	0.05	0
	1	77	0.10	0.02

	2	77	0.05	0.02*2
	3	77	0.10	0.02*3
	4	77	0.05	0.02*4
	5	77	0.10	0.02*5
Burst 2	Same as Burst 1 (RESET_MODE = 2)			

Example Illustration - 2

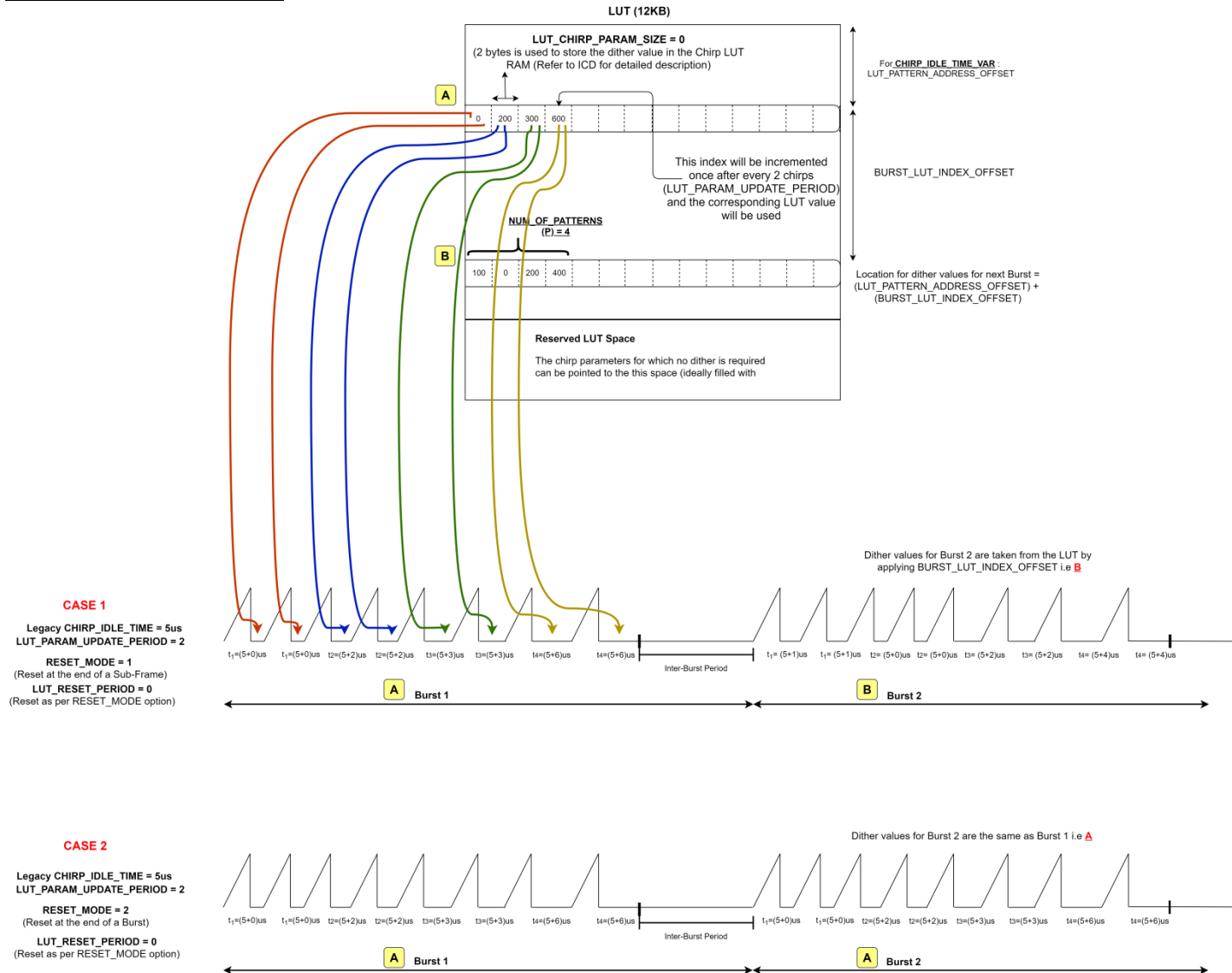


Figure 9. Advanced chirp – Example Illustration 2

This figure considers the variation of just one Chirp Parameter i.e. *CHIRP_IDLE_TIME_VAR*. As only Idle time Variation is considered in this example, all the rest of the chirp parameters point to the reserved LUT Space (filled with zeros).

For *CHIRP_IDLE_TIME_VAR*, only LUT Dither is considered in this example.

Let's Consider Pattern A first. As it is evident from the above figure, and working again like previous figure, we can infer:

- a. *LUT_PATTERN_ADDRESS_OFFSET* = *X*
- b. *NUM_OF_PATTERNS* (*P*) = 4 as four values are programmed for this chirp parameter, corresponding to "0", "200", "300" and "600". (1 LSB = 10ns, Hence, the dithers correspond to 0us, 2us, 3us and 6us)
- c. It can be seen from the figure that each of the four colors (red, blue, green and yellow) point to two chirps simultaneously. The chirp parameter is updated with the new value from LUT after every 2 chirps, hence *LUT_PARAM_UPDATE_PERIOD* (*K*) = 2 in this case.
- d. *LUT_CHIRP_PARAM_SIZE* = 0, which corresponds to a size of 2 bytes in the LUT (Refer to ICD)
- e. *LUT_CHIRP_PARAM_SCALE* is not used in this case; hence its value is 0.

CASE 1:

Here *RESET_MODE* = 2, i.e. reset at the end of every burst and the *LUT_RESET_PERIOD* = 0, which points to the default reset mode only (assuming a burst is considered which has only 8 configured chirps). As it can be seen from the figure, the dither values for burst 2 are the same as burst 1 i.e. pattern A.

CASE 2:

Now as it is noticed, there are two idle time dither rows in the LUT. The lower row in the LUT corresponds to pattern B.

Here *RESET_MODE* = 1, i.e. reset at the end of every sub-frame and the *LUT_RESET_PERIOD* = 0, which points to the default reset mode only (assuming a burst is considered which has only 8 configured chirps). Let's assume there are 2 bursts in this sub-frame, because reset happens only at the end of a sub-frame, both these bursts would be different.

Pattern B corresponds to the idle time dither values for Burst 2:

- a. Location of dither values for the next burst = *LUT_PATTERN_ADDRESS_OFFSET* + *BURST_LUT_INDEX_OFFSET* = *X+Y*. Parameter *BURST_LUT_INDEX_OFFSET* of this API needs to be set to 'Y' so at every next burst of sub-frame, dither offset is added by 'Y'.
- b. *NUM_OF_PATTERNS* (*P*) = 4 as four values are programmed for this chirp parameter, corresponding to "100", "0", "200" and "400". (1 LSB = 10ns, Hence, the dithers correspond to 1us, 0us, 2us and 4us)

- c. Again, the chirp parameter is updated with the new value from LUT after every 2 chirp, hence $LUT_PARAM_UPDATE_PERIOD (K) = 2$.
- d. $LUT_CHIRP_PARAM_SIZE = 0$, which corresponds to a size of 2 bytes in the LUT (Refer to ICD)
- e. $LUT_CHIRP_PARAM_SCALE$ is not used in this case; hence its value is 0.

Now looking at the chirp diagram for Case 2, we can infer that burst 1 uses Pattern A, whereas burst 2 of the sub-frame uses pattern B.

This way, by configuring different *RESET MODES*, we can have different dithered bursts or sub-frames.

Limitation

- The first chirp in a burst (AFC) or in a legacy frame shall be discarded due to HW limitation in below cases:
 - If start frequency dither is negative for 2nd chirp of a burst/frame (either due to delta increment or due to LUT value).
Workaround: program a small negative dither for 1st chirp in LUT.
 - If start frequency dither is $\geq \pm 450\text{MHz}$ for 2nd chirp of a burst/frame (either due to delta increment or due to LUT value).
Workaround: Discard 1st chirp data.
 - If slope dither is negative for 2nd chirp of a burst/frame (either due to delta increment or due to LUT value).
Workaround: program a small negative dither for 1st chirp in LUT.
 - If slope dither is $\geq \pm 3\text{MHz/us}$ for 2nd chirp of a burst/frame (either due to delta increment or due to LUT value).
Workaround: Discard 1st chirp data.
- The minimum chirp duration or cycle time shall be 25us if advance chirp feature is used (vs 13us in case of legacy chirp configuration API is used).

Following figures depict the API flow to configure different parameters of advanced chirp. Please refer the DFP examples to find out more.

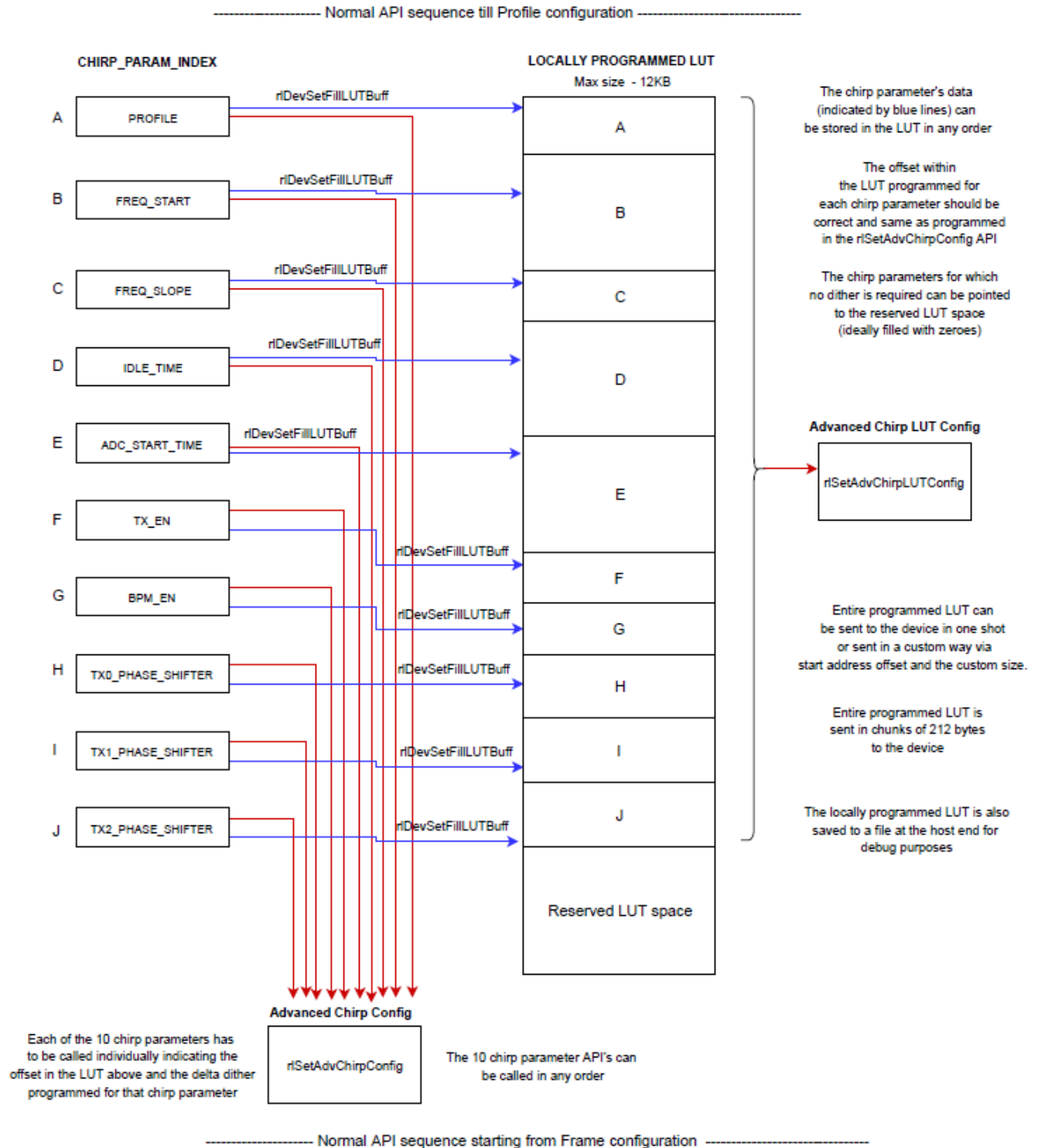


Figure 10. Advanced chirp – Big picture

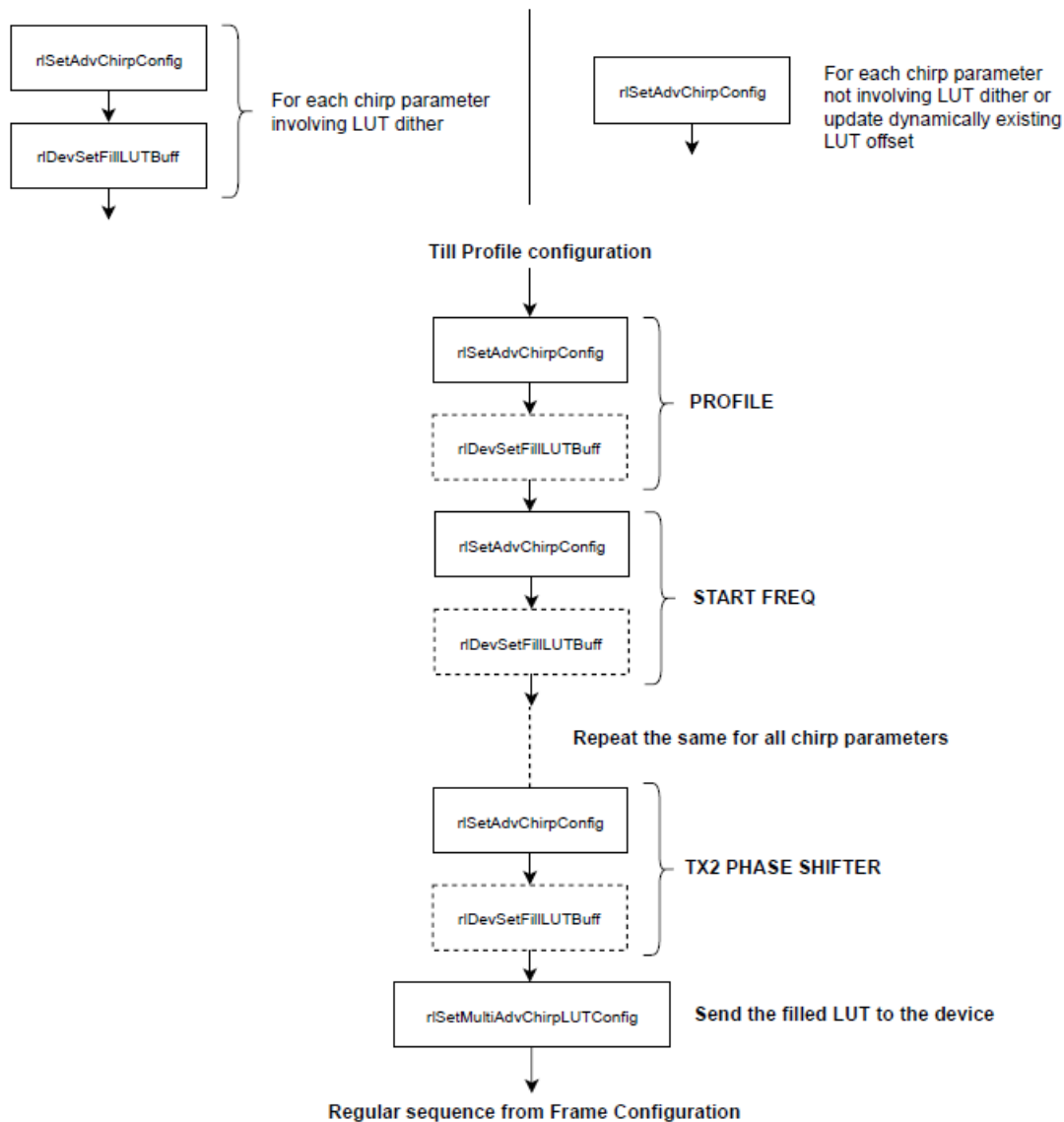


Figure 11. Advanced chirp – Flow diagram

6.2.1 Dynamic Dither (delta/LUT) Update

If at every next frame few of chirp parameters need to be updated dynamically then application needs to set LUT with new data at different offset (`riSetAdvChirpLUTConfig`) and invoke `riSetAdvChirpConfig` to connect this parameter to new offset (`lutPatternAddressOffset`). And on top of that it needs to call `riSetDynChirpEn` API which actually applies new changes for next frame. All these three APIs must need to be called within current active-frame boundary only then new LUT dither will be applied to next frame.

6.3 Dynamic chirp configuration

There are scenarios where different chirp parameters need to update on the go. Updating the Chirp RAM is possible using `rlSetChirpConfig` API but this API is slow compare to frame periodicity to apply the changes which may break the device flow.

`rlSetDynChirpCfg` API can be used to dynamically change the chirp configuration while frames are on-going without much delay. The configuration will be stored in software using `rlSetDynChirpCfg` API and at `rlDynChirpEnCfg` API invocation, RadarSS copies these chirp configurations from SW RAM to HW RAM at the end of current on-going frame.

This API provides much flexibility to accommodate different type of chirp parameters of dissimilar chirp index/segment within single API which effectively speed up the configuration time for whole chirp RAM (512 unique chirps). This API gives direct access to device software Chirp RAM which is divided into different segments contains set of chirps. Each chirp in the RAM is divided into three rows.

Chirp row 1: Profile index, frequency slope, Tx enable, BPM constant.

Chirp row 2: frequency start var.

Chirp row 3: idle time, ADC start time.

If user wishes to reconfigure the entire chirp parameters i.e. fill all the chirp rows, then single `rlSetDynChirpCfg` API can configure 16 unique continuous chirps on the device RAM. This way it takes only 32 API calls to configure whole chirp RAM.

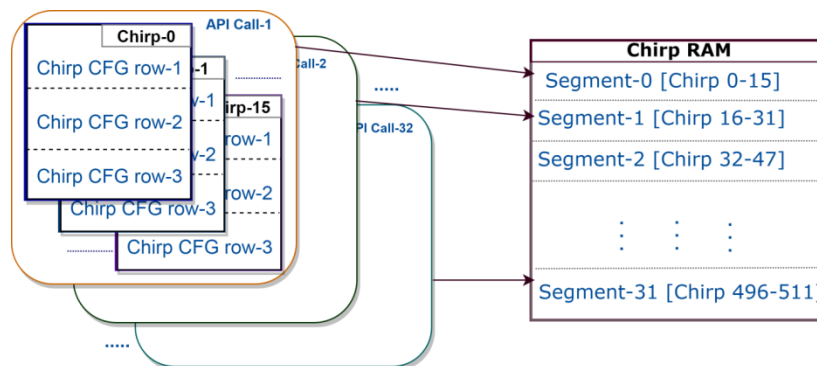


Figure 12. Chirp row select = 0

If user does not wish to reconfigure all 3 chirp rows, then the following mode can be used to configure only one row per chirp which enables the user to configure 48 chirps in one API, effectively saving on the reconfiguration time.

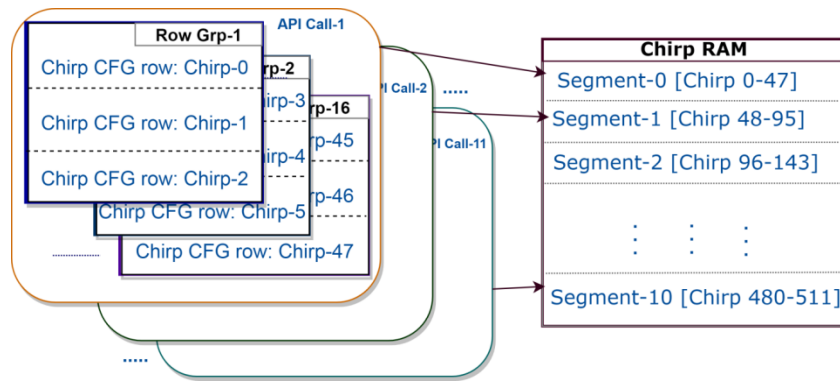


Figure 13. Chirp row select = 1/2/3. (Not using legacy chirp at the beginning)

All these chirp configurations are stored to software chirp RAM, to apply to HW chirp RAM device provides two programming modes.

In program mode 0, a separate API `rilSetDynChirpEn` needs to be called where device will re-configure the HW only at the end of current active frame boundary. In this case, application can invoke `rilSetDynChirpCfg` at any time during the frame and those chirp configurations will be stored at software memory but `rilSetDynChirpEn` API should be called within active frame time.

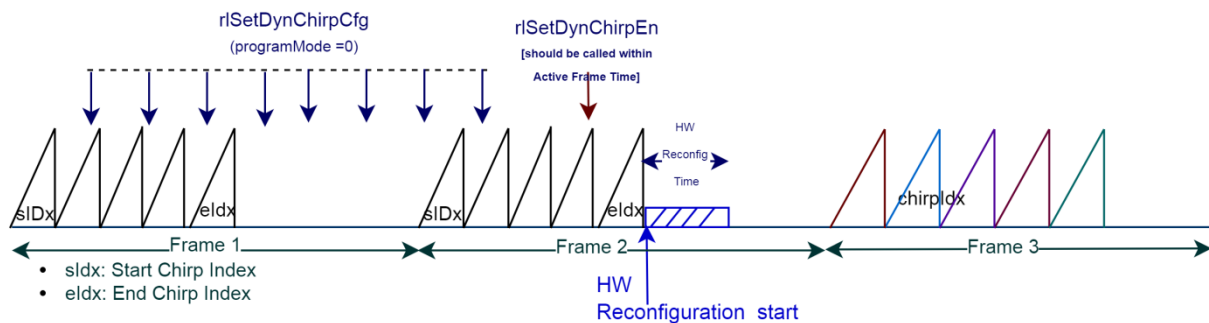


Figure 14. Program mode = 0

In program mode 1, new chirp configurations are applied to HW at the same time of `rilSetDynChirpCfg` API call.

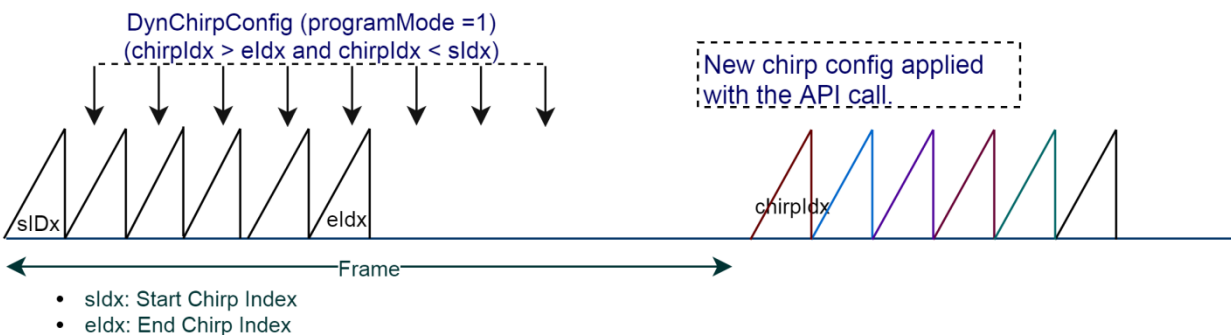
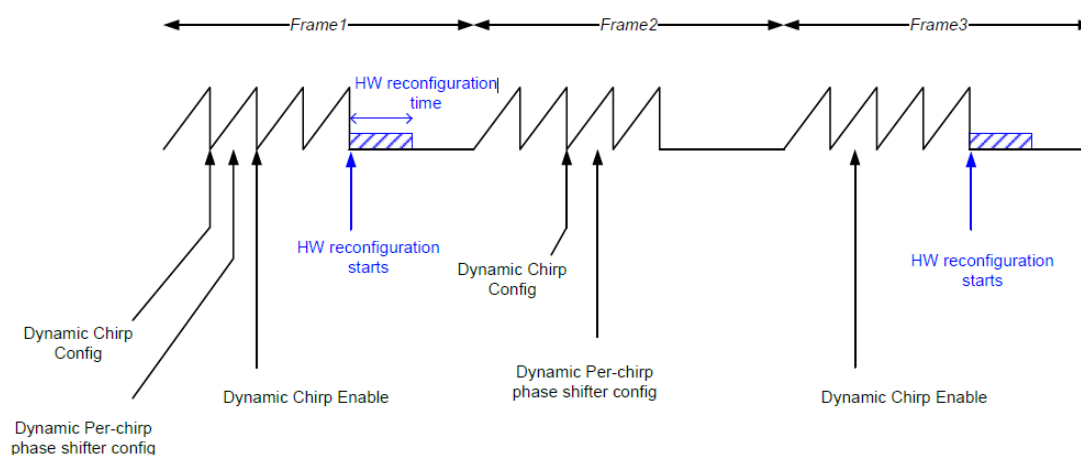


Figure 15. Program mode = 1. (Program the new configuration immediately)
NOTE:

1. User has to ensure that the chirps which are being reconfigured are not the ones which are being used by current running frame.
2. If used the legacy chirp configuration at the beginning, then dynamic chirp must have at least those chirp indices which were configured by legacy chirpConfig API
3. HW reconfiguration time (as shown in the figure below) is around 500 μ s. User has to ensure that `rlSetDynChirpEn` API is issued at least 500 μ s before the end of the ongoing frame active window (start of the frame idle time) to apply configurations for next frame onwards.


Figure 16. Dynamic Chirp use case timing diagram

6.4 Dynamic profile configuration

Profile configuration can be updated in the sensor during run time using the `rlSetProfileConfig` API. When this API is being called during framing, new parameters will be stored in the RAM which will be applied to hardware around frame end boundary. New profile parameters will be visible to the next frame.

TX calibration LUT needs to be updated if TX output power back off value is changed in the dynamic profile configuration. RadarSS updates the LUT for RX/TX at the beginning of next frame if requested in dynamic profile configuration. If application doesn't require to update LUT, then set `[pfCalLutUpdate=0x3]` in profile configuration API. It is recommended not to request for update LUT in profile configuration while updating other parameters in this API dynamically.

6.5 Dynamic per chirp phase shifter configuration

Phase shift to each TX can be configured in the mmWave sensor using either `rlSetProfileConfig` or `rlRfSetPhaseShiftConfig` API. Profile configuration provides an option to add phase shift for specific profile index, so the chirps mapped to this profile index will have phase shift added to configured TX.

But if application requires adding phase shift to TX for each of chirp or group of chirps (independent of profile index), then use `rlRfSetPhaseShiftConfig` API. With this API, group of chirp can be selected to set phase shift value for TX [0-2] at the unit of 5.625°.

This API can be used to dynamically change the per-chirp phase shifter configuration (applicable only in certain devices) while frames are on-going. The configuration will be stored in software and the new configuration will be applied after receiving the `rlSetDynChirpEn` API.

If phase shifter is used by the per-chirp-phase-shifter, then RadarSS uses phase shifter value only from this API and not from Profile-phase-shifter. On the other hand, if the profile phase-shifter is set & not in the per-chirp, then RadarSS uses phase shifter value only from the profile configuration API.

6.6 Factory Calibration

mmWave Sensor radar system on chip includes self-calibrations to mitigate process and temperature effects on analog performance. The calibrations include RF INIT (i.e., boot time) calibrations to mitigate manufacturing process variation effects, and Run Time calibrations to mitigate temperature effects. These calibrations mostly involve optimizing the RF register settings for TX, RX and LO based on the temperature read by the built-in temperature sensors.

This allows the devices to self-calibrate to account for manufacturing process variation. The ambient temperature (called factory calibration temperature) may be 25°C or the sensors typically expected in-field temperature. The factory calibrations should be done at the RF frequency range of interest for the sensor.

The following are recommended to be done in-field to ensure that all devices are restored to the same state as during Tier 1 factory calibration. This can be done using the following procedure. It has slight deviations from the normal RF INIT call and start up sequence.

Note that since selected calibrations are disabled during calibration restore, the calibration report API messages (AWR AE RF INITCALIBSTATUS SB) will also report those calibration statuses as 0. Note also that the mandatory calibrations (e.g. to keep the PLLs in lock) are anyway done without host control.

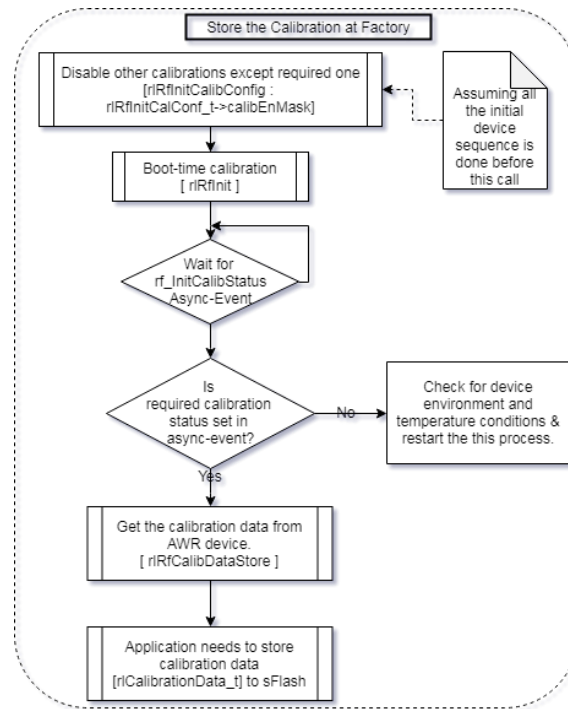


Figure 19. Calibration Store – flow diagram

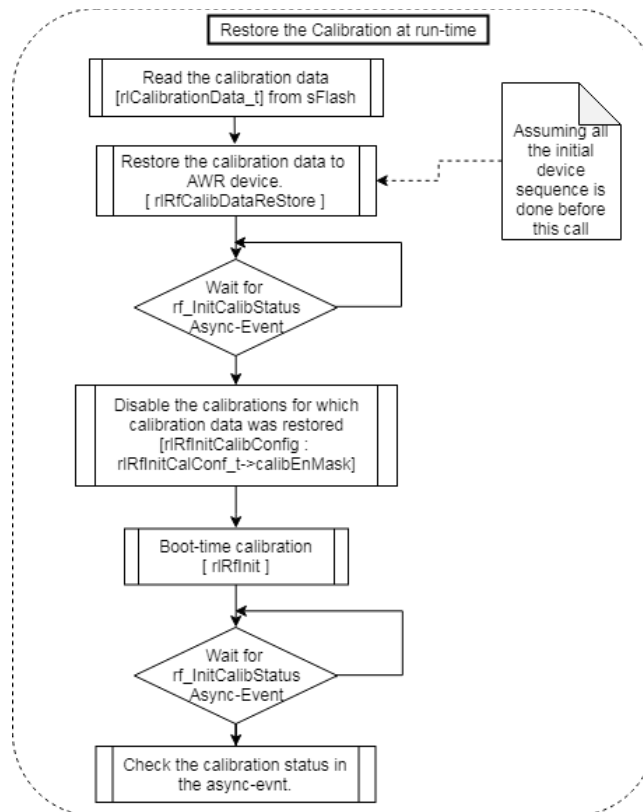


Figure 20. Calibration Restore – flow diagram

NOTE:

1. While capturing the calibration data, it should be done in the controlled environment.
2. No other Radar device or signal to cause interference to the DUT (device under test).
3. Device environment should be at room temperature.

6.7 CP/CQ Data Extraction and Verification for xWR294x/xWR254x/xWR2x4xP/xWR2x4xECO/xWR2x4xLC devices

The code flow will be as follows

1. Configure the EDMA channel which will be used to transfer data from the CQ RAM to the L3 Memory of the device for every chirp in a frame which can then be processed as required.
2. Configure the ADCBUF data format and other configurations required for CQ data.
3. Configure the RX Saturation Monitor
4. Enable the Monitor
5. Trigger the EDMA manually at the start of every chirp in the Chirp available ISR to collect data for the previous chirp

Step 1) Configure the EDMA as in the below code snippet. Please note that CQ2 data max can be 128 bytes for a single chirp as per the max number of time slices we can have. Currently we are only collecting 64 bytes of the data as we have set the time slices to a total of 64. The same value is used for the ACNT parameter. For CCNT we are using the total number of chirps in our frame. Our frame had 128 chirps hence we have set it to 128. Change this value accordingly as per your use case. Use this address values for

- CQ2_RAM_ADDRESS-----0xA5010400U
- L3_MEM_ADDRESS-----0x88000000U
- CP_MEM_ADDRESS-----0x05020C08U

```
/* initialize and configure eDMA to copy CQ Data to L3 memory */
int MmwaveLink_CQedmaSetup(void)
{
    //Edma_IntrObject  intrObj;
    baseAddr = EDMA_getBaseAddr(gEdmaHandle[0]);
    DebugP_assert(baseAddr != 0);

    regionId = EDMA_getRegionId(gEdmaHandle[0]);
    DebugP_assert(regionId < SOC_EDMA_NUM_REGIONS);

    dmaCh0 = EDMA_RESOURCE_ALLOC_ANY;
    Status = EDMA_allocDmaChannel(gEdmaHandle[0], &dmaCh0);
}
```

```

DebugP_assert(Status == SystemP_SUCCESS);

    tcc0 = EDMA_RESOURCE_ALLOC_ANY;
    Status = EDMA_allocTcc(gEdmaHandle[0], &tcc0);
    DebugP_assert(Status == SystemP_SUCCESS);

    param0 = EDMA_RESOURCE_ALLOC_ANY;
    Status = EDMA_allocParam(gEdmaHandle[0], &param0);
    DebugP_assert(Status == SystemP_SUCCESS);

    /* Request channel */
    EDMA_configureChannelRegion(baseAddr, regionId, EDMA_CHANNEL_TYPE_DMA,
        dmaCh0, tcc0, param0, 0);

    /* Program Param Set */
    EDMACCPaRAMEntry edmaParam;
    EDMA_ccPaRAMEntry_init(&edmaParam);
    edmaParam.srcAddr    = (uint32_t)CQ2_RAM_ADDRESS;
    edmaParam.destAddr   = (uint32_t)L3_MEM_ADDRESS;
    edmaParam.aCnt       = (uint16_t) 64U; //CQ2 data is 128 max
    edmaParam.bCnt       = (uint16_t) 1U;
    edmaParam.cCnt       = (uint16_t) 128U;
    edmaParam.bCntReload = (uint16_t) 1U;
    edmaParam.srcBldx    = (int16_t) 0U;
    edmaParam.destBldx   = (int16_t) 0U;
    edmaParam.srcCldx    = (int16_t) 0U;
    edmaParam.destCldx   = (int16_t) 64U;
    edmaParam.linkAddr   = 0xFFFFU;
    edmaParam.opt        |=
        (EDMA_OPT_TCINTEN_MASK | EDMA_OPT_ITCINTEN_MASK |
        (((uint32_t)tcc0) << EDMA_OPT_TCC_SHIFT) & EDMA_OPT_TCC_MASK));

    EDMA_setPaRAM(baseAddr, param0, &edmaParam);
    return 0;
}

```

Step 2) Configure the ADCBUF as per the code snippet below. You can enable all the channels or in any combination as per your use case.

```

int32_t MMwaveLink_AdcBufConfigCQ()
{
    int status=0;
    ADCBuf_Handle adchandle;
    adchandle=ADCBuf_open(0,NULL);
}

```

```

ADCBuf_CQConf cqconfig;
ADCBuf_dataFormat dataformat;
ADCBuf_RxChanConf chanconfig={0};
chanconfig.channel=0;
dataformat.adcOutFormat=1;
dataformat.sampleInterleave=0;
dataformat.channelInterleave=1;
cqconfig.cqDataWidth=1;
cqconfig.cq96BitPackEn=0;
cqconfig.cq0AddrOffset=0x0U;
cqconfig.cq1AddrOffset=0x200U;
cqconfig.cq2AddrOffset=0x400U;
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CONF_CQ,(void*)&cqconfig);
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CONF_DATA_FORMAT,(void*)&dataformat);
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CHANNEL_ENABLE,(void*)&chanconfig);
chanconfig.channel=1;
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CHANNEL_ENABLE,(void*)&chanconfig);
chanconfig.channel=2;
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CHANNEL_ENABLE,(void*)&chanconfig);
chanconfig.channel=3;
status=ADCBuf_control(adchandle,ADCBufMMWave_CMD_CHANNEL_ENABLE,(void*)&chanconfig);
return status;
}

```

Step 3) Configure the RX Saturation Monitor with the below parameters.

```

const rIRxSatMonConf_t rxSatMonCfg =
{
    .profileIndx    = 0U,
    .satMonSel      = 3U,
    .reserved0      = 0U,
    .primarySliceDuration = 5,
    .numSlices      = 63, //CQ_NUM_SLICES,
    .rxChannelMask  = 0U,
    .reserved1      = 0U,
    .reserved2      = 0U,
    .reserved3      = 0U,
    .reserved4      = 0U
};

```

Step 4) Enable the 24th bit of the Analog monitor for the Saturation monitor.

```
int32_t MmwaveLink_setRfAnaMonConfigCq (void)
{
    int32_t retVal;

    (0 << 2) | \
    (0 << 3) | \
    (0 << 4) | \
    (0 << 5) | \
    (0 << 6) | \
    (0 << 7) | \
    (0 << 8) | \
    (0 << 9) | \
    (0 << 10) | \
    (0 << 11) | \
    (0 << 12) | \
    (0 << 13) | \
    (0 << 14) | \
    (0 << 15) | \
    (0 << 16) | \
    (0 << 17) | \
    (0 << 18) | \
    (0 << 19) | \
    (0 << 20) | \
    (0 << 21) | \
    (0 << 22) | \
    (0 << 23) | \
    (1 << 24) | \
    (0 << 25) | \
    (0 << 26) | \
    (0 << 27) | \
    (0 << 28) | \
    (0 << 29) | \
    (0 << 30) | \
    (0 << 31)
    ,0x0};

    /* Set analog monitor enable configuration */
    retVal = rIRfAnaMonConfig(RL_DEVICE_MAP_INTERNAL_BSS,&data);
}
```

Step 5) Trigger the EDMA for every Chirp available ISR. Please note that for the last chirp in the frame you need to trigger the EDMA separately as this will not capture the data for the last chirp of the frame. We collected the last chunk of data separately in the code after sensor stop command.

```
void MmwaveLink_triggerEdmaTransfer()
{

    EDMA_enableTransferRegion(baseAddr, regionId, dmaCh0,
        EDMA_TRIG_MODE_MANUAL);
    while(EDMA_readIntrStatusRegion(baseAddr, regionId, tcc0) != 1)
        ;

    EDMA_clrIntrRegion(baseAddr, regionId, tcc0);
    eDMADataTransferDone++;
}

/**
 * @b Description
 * @n
 * Interrupt handler callback for chirp available ISR.
 *
 * @retval
 * Not Applicable.
 */

void Mmwavelink_chirpAvailInterrupCallBackFunc(void* arg)
{
    /* CQ data verification test: trigger eDMA to copy data from CQ RAM to L3 mem.
     * Note - this test has been defined for legacy frame scenario, for advanced frame config user needs
     to write seperate logic.
     */
    if (cqDataVerifyTest && gLinkChirpCnt>0)
    {
        /* Trigger eDMA to trasnfer CQ2 RAM data to L3 memory for each chirp */
        MmwaveLink_triggerEdmaTransfer();
    }
    /* increment the Chirp count for current frame */
    gLinkChirpCnt++;
}
```

You should see the CQ RAM with all the CQ2 Data as below.

```
0xA50103E0 DEADE3DE F02DCC51 CF762221 BB8E0C52 9D102FB7 19899E35 BA51113B A7051A53
0xA5010400 0000003F 00000000 00000000 00000000 00000000 00000000 00000000
0xA5010420 00000000 00000000 00000000 00000000 00000000 00000000 00000000
0xA5010440 847B6DAB 11128B1B B9E9B06C AA669916 9BFB4B6D 16F3C6D4 714E6134 7946929A
0xA5010460 B1182988 B617DEFE 6E47D7F3 18308BE1 E8E71025 68159D33 D28796F8 1F879D77
0xA5010480 FEA3EEEE FE1DC65A CF6F483F B225703C 40A9FD5E F4CFAB44 E722D97A F198A3B8
0xA50104A0 7B50E774 87DA1873 EE734DFF 1FDBAE36 4C879238 57AE22CE 1674A7EB 37E1DBA7
0xA50104C0 6611124E 0D7FF72E F00CB8AC 9B516E39 752031A5 D0B04E18 674DD343 656873AB
```

Figure 21. CQ2 Memory Location no interference condition

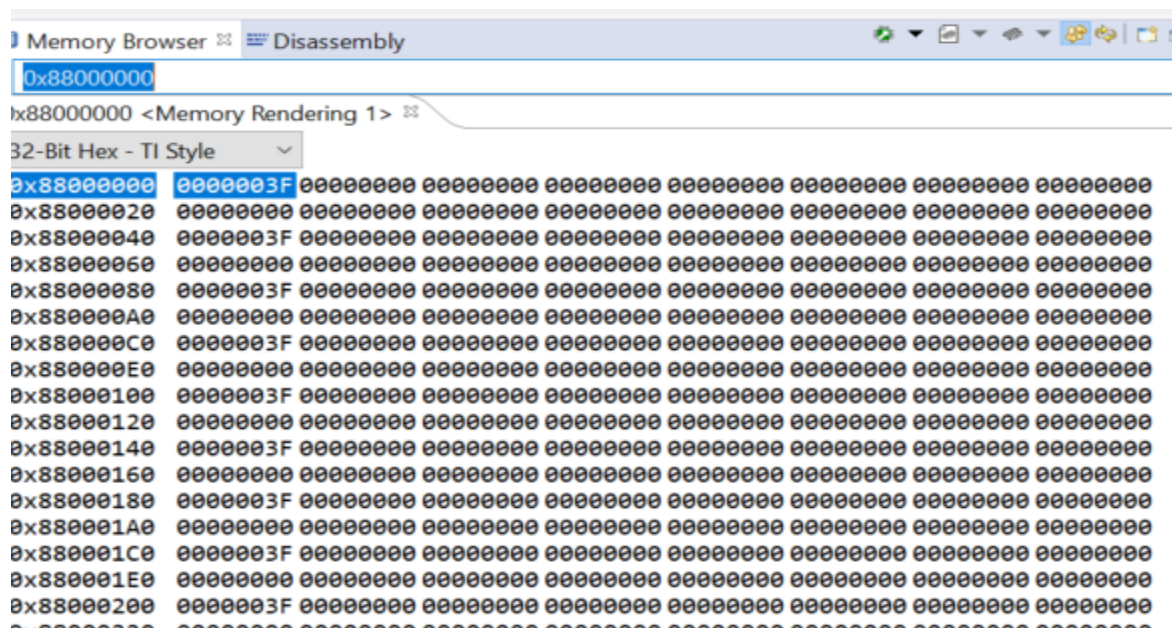


Figure 22. L3 Memory Location

L3 ram is shown as above. Currently you should see all zeros as we had no saturation events. If you provide strong interference to the device you should observe non-zero values.

Also, below we have tested the board on the bench with a proper setup to introduce interference and were able to see RX saturation data.

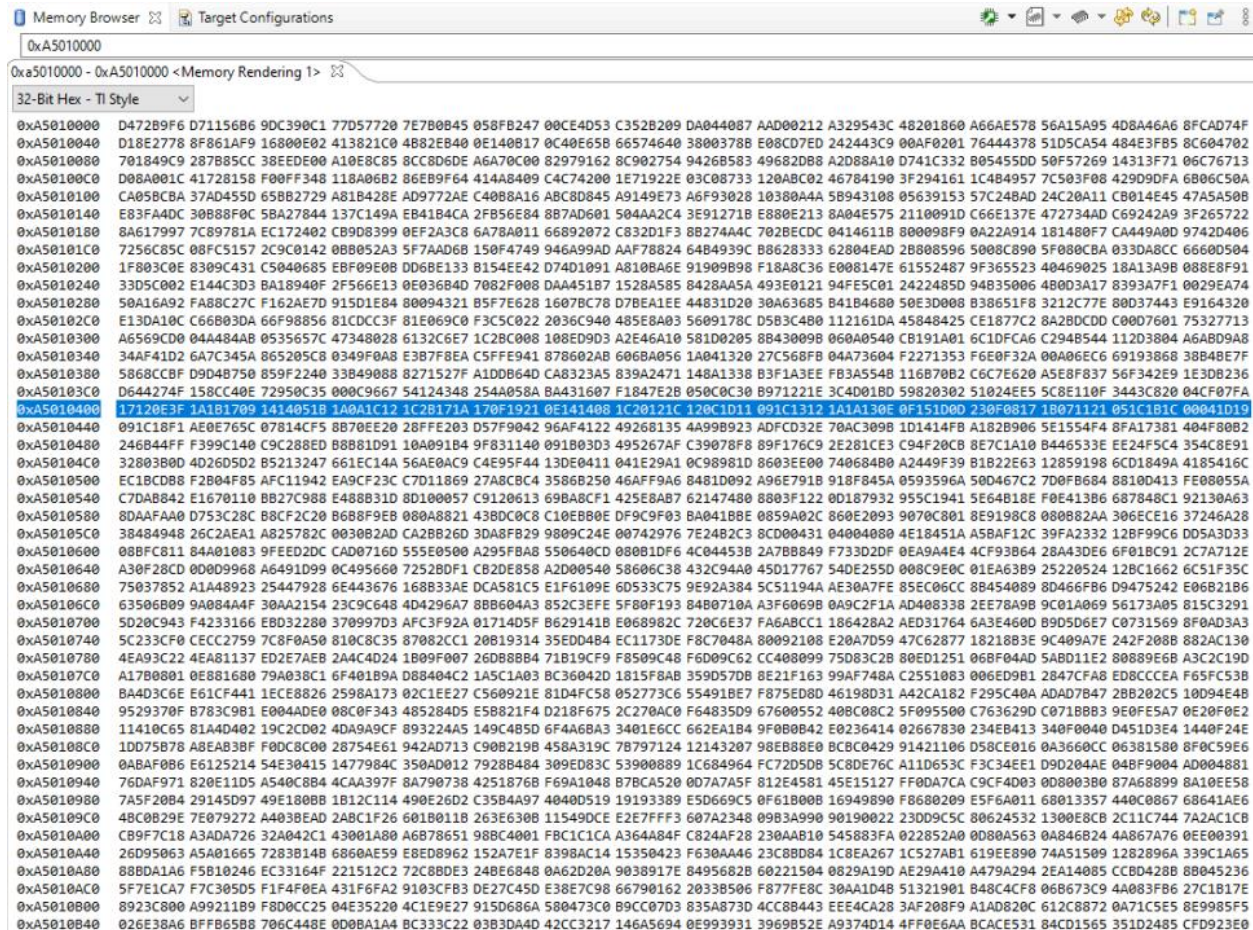


Figure 23. CQ2 Memory Location with Interference

The non-zero values in the CQ2 memory location ensures that our device was able to detect the saturation events in case an interfering signal is present.

7. Platform and Device Support

The device and platforms supported with this release include:

Supported Devices	Release Status	Supported EVMs
xWR294xECO ES1.0	Production Release	xWR294xECO EVM
xWR2E4xECO ES1.0	Production Release	xWR2E4xECO EVM
xWR294xP ES1.0	Production Release	xWR294xP EVM
xWR2E4xP ES1.0	Production Release	xWR2E4xP EVM
xWR254x ES1.0	Production Release	xWR254x EVM
xWR294x ES2.0	Production Release	xWR294x EVM
xWR2x4xLC ES1.0	Production Release	xWR2x4xP EVM